

# Policy Iteration with Human Feedback: Bringing Post-Training RL to In-context Learning

Minh-Ha Nguyen<sup>1</sup> and Cathy Shyr<sup>2,3,4</sup>

<sup>1</sup>*Department of Epidemiology, Vanderbilt University, Nashville, TN, USA*

<sup>2</sup>*Department of Pediatrics, Vanderbilt University Medical Center, Nashville, TN, USA*

<sup>3</sup>*Department of Biostatistics, Vanderbilt University Medical Center, Nashville, TN, USA*

<sup>4</sup>*Department of Biomedical Informatics, Vanderbilt University Medical Center, Nashville, TN, USA*

16 August 2026

## Abstract

Generative pretraining established reusable task representations; later work on language-based task conditioning and in-context learning showed that a fixed model could adapt its behavior from instructions and demonstrations. Policy Iteration with Human Feedback (PIHF) builds on this development and the recurrent evaluate-and-improve structure of generalized policy iteration. PIHF uses a pretrained language model as its execution substrate and moves persistent revision to a versioned natural-language policy and tool set. A language-model critic and clinical expert review complete-panel reasoning and tool-use trajectories to localize recurrent failures and form candidate revisions; the expert may reinterpret the evidence and retains authority over admission and rollback, while Recall@1 and Recall@5 validate outcomes after candidate execution.

Across cumulative ablations and ultra-rare-disease benchmarks, a PIHF-derived policy improved Recall@1 in one proprietary executor and three open-weight executors spanning 3 to 49 billion active parameters. Gains were 32.7 percentage points for GPT-5.4 and 31.1 points for Qwen3.6-35B, a difference of 1.7 points. These results support the feasibility of using pretrained language models as fixed-weight execution substrates for expert-guided policy development in rare-disease diagnosis.

## 1 Reinforcement-learning principles

Reinforcement learning represents behavior by a policy and improves that policy by increasing expected return [1]. For language-model feedback learning,  $x \sim \mathcal{D}$  is a prompt sampled from a prompt distribution, and  $y \sim \pi_\theta(\cdot | x)$  is a response sampled from the current policy. The scalar  $R(x, y)$  evaluates that prompt-response pair. Ziegler et al. and InstructGPT use a per-response log-ratio penalty (Equation 2 in each paper, with InstructGPT’s pretraining term omitted). Writing the expected log ratio as a KL divergence gives the objective stated directly by Rafailov et al. in their Equation 3 [2, 3, 4]:

$$J(\theta) = \mathbb{E}_{\substack{x \sim \mathcal{D} \\ y \sim \pi_\theta(\cdot | x)}} [R(x, y)] - \beta \mathbb{E}_{x \sim \mathcal{D}} [\text{KL}(\pi_\theta(\cdot | x) \| \pi_{\text{anc}}(\cdot | x))], \quad \beta > 0. \quad (1)$$

The first term is the expected evaluator score of responses produced by the current policy and supplies the reward-seeking component of policy improvement. The second term penalizes

departure from the anchor policy  $\pi_{\text{anc}}$ , and  $\beta$  sets the strength of that penalty. The evaluator  $R$  maps each prompt–response pair to the scalar signal used by the first term; in language-model feedback learning, it may be a learned reward model, a rule-based evaluator, or a human-derived score.

The notation separates three kinds of variation. Sampling variation comes from the prompts  $x$  and responses  $y$  observed across rollouts. Learning changes  $\theta$  and therefore changes the conditional response distribution  $\pi_{\theta}(\cdot | x)$ . Within one declared policy-update phase, the prompt distribution  $\mathcal{D}$ , the evaluator  $R$ , the anchor  $\pi_{\text{anc}}$ , and the coefficient  $\beta$  are held fixed. Across tasks or training phases,  $\mathcal{D}$  and  $R$  may change and thereby define a new environment and evaluation target.

The objective has a closed-form exponential-tilt optimizer. Rafailov et al. give this language-model preference-optimization form in their Equation 4 [4].

**Proposition 1** (KL-regularized response tilt). *For a fixed  $x$ , suppose  $0 < Z_{\beta}(x) < \infty$ . Maximizing the corresponding integrand of Equation 1 over conditional distributions supported by  $\pi_{\text{anc}}$  yields*

$$\pi^*(y | x) = \frac{\pi_{\text{anc}}(y | x) \exp\{R(x, y)/\beta\}}{Z_{\beta}(x)}, \quad Z_{\beta}(x) = \sum_{y'} \pi_{\text{anc}}(y' | x) \exp\{R(x, y')/\beta\}. \quad (2)$$

Equation 2 makes the improvement mechanism explicit. The anchor supplies the base response distribution, while  $\exp\{R(x, y)/\beta\}$  reweights each response according to its evaluated quality. The normalizer  $Z_{\beta}(x)$  converts those reweighted masses into a conditional distribution. Higher reward therefore shifts probability toward a response relative to other responses for the same prompt.

Applying the score-function identity underlying REINFORCE [5, Section 4, Theorem 1] to the full regularized objective, and using  $\mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)}[\nabla_{\theta} \log \pi_{\theta}(y | x)] = 0$ , gives

$$\nabla_{\theta} J = \mathbb{E}_{\substack{x \sim \mathcal{D} \\ y \sim \pi_{\theta}(\cdot | x)}} \left[ \left( R(x, y) - \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{anc}}(y | x)} - b(x) \right) \nabla_{\theta} \log \pi_{\theta}(y | x) \right], \quad (3)$$

where  $b(x)$  is any prompt-dependent, response-independent baseline. It leaves the expected gradient unchanged and can be chosen to reduce variance. The quantity in parentheses is the baseline-adjusted learning signal:  $R(x, y)$  contributes evaluated quality, and the log-ratio contributes anchor pressure. The score term  $\nabla_{\theta} \log \pi_{\theta}(y | x)$  converts that signal into changes in response probability. Together, policy evaluation and this update instantiate the recurring evaluate-and-improve pattern of reinforcement learning.

## 2 Structural bridge to PIHF

Generalized policy iteration organizes reinforcement learning as an interaction between policy evaluation and policy improvement [1, Section 4.6]. In weight-space language-model reinforcement learning, rollouts from  $\pi_{\theta}$  expose current behavior,  $R(x, y)$  evaluates sampled prompt–response pairs, and optimization changes  $\theta$  so that subsequent rollouts place more probability on higher-valued responses under the regularized objective.

PIHF implements corresponding functions over an external artifact. Complete-panel trajectories and outcomes provide evaluation evidence. The critic and expert use that evidence to localize recurrent failures and form targeted revisions; candidate freeze, complete-panel comparison, and expert admission determine which artifact persists to the next iteration.

The correspondence first separates policy representation from induced behavior. In weight-space reinforcement learning,  $\theta$  denotes the learned model parameters. In PIHF,  $t$  indexes the

iteration and  $A_t = (P_t, T_t)$  denotes the external artifact, with natural-language policy  $P_t$  and available tools  $T_t$ . The representation-level correspondence is

$$\theta \longleftrightarrow A_t = (P_t, T_t). \quad (4)$$

Given a prompt  $x$ ,  $\pi_\theta(y | x)$  denotes the distribution over responses  $y$ . Given a clinical case  $z$ ,  $\pi_{M, A_t}(\tau | z)$  denotes the distribution over complete trajectories  $\tau$  induced when the frozen executor  $M$  applies  $A_t$  to  $z$ . The behavior-level correspondence is

$$\pi_\theta(y | x) \longleftrightarrow \pi_{M, A_t}(\tau | z). \quad (5)$$

At the sample level, the prompt and response correspond to the case-instantiated prompt and complete trajectory:

$$x \longleftrightarrow x_t(z), \quad (6)$$

$$y \longleftrightarrow \tau. \quad (7)$$

PIHF separates two functions that a scalar reward may combine in weight-space RL. First, critic and expert review of panel trajectories localizes a recurrent failure to the policy stage or tool behavior that should change. This process feedback supplies credit assignment for the discrete artifact revision. Second, Recall@1 and Recall@5 measure terminal diagnostic outcomes after the frozen candidate executes the complete panel. Proposal formation, candidate freeze, outcome validation, and expert admission therefore implement the improvement cycle while keeping process feedback and outcome validation as distinct evidence streams.

### 3 In-context policy definition

**Definition 1** (In-context policy representation). Fix an executor model  $M$ . At iteration  $t$ , the mutable representation is

$$A_t = (P_t, T_t), \quad (8)$$

where  $P_t$  is the versioned natural-language policy and  $T_t$  is the set of tools available to the executor.

For a clinical case  $z$ , the policy instantiates the executor prompt

$$x_t(z) = \text{Prompt}(P_t, z). \quad (9)$$

The symbol  $\tau$  denotes one complete rollout trajectory:

$$\tau = (e_1, \dots, e_L, \hat{y}), \quad (10)$$

where  $e_\ell$  is a recorded model output, tool call, or tool result, and  $\hat{y}$  is the final ranked differential. The artifact induces the trajectory distribution

$$\pi_{M, A_t}(\tau | z) := p_M(\tau | x_t(z); T_t). \quad (11)$$

A complete trajectory may therefore span several model calls connected by intervening tool calls and results.

The policy organizes execution into ordered stages

$$\Phi_t = (\phi_t^{(1)}, \dots, \phi_t^{(m_t)}). \quad (12)$$

The stages instruct execution and index review. A critique localizes the earliest stage where  $\tau$  departs from  $P_t$  and attributes the departure to a policy rule, tool use, or returned evidence. Conformance asks whether  $\tau$  followed  $P_t$ . Adequacy asks whether  $P_t$  remains clinically sound and useful, using clinical evidence, panel outcomes, critic analysis, and expert judgment.

*RL parallel.* The artifact  $A_t$  is the policy representation, and  $\pi_{M,A_t}(\tau \mid z)$  is its induced behavior.

## 4 Pretrained representations and in-context adaptation

Generative pretraining produced representations that could be reused across language tasks. Radford et al. demonstrated this transfer by expressing diverse tasks through task-aware sequence interfaces and adapting a shared pretrained Transformer through supervised fine-tuning [6]. Their subsequent GPT-2 study expanded the role of reuse: language could encode the task, input, and output in one sequence and condition model behavior under fixed parameters [7]. Brown et al. then defined this fixed-weight inner loop as in-context learning, in which instructions, demonstrations, or both specify a task within the input sequence at inference [8]. This research line moved from adapting reusable representations through fine-tuning to eliciting task-conditioned behavior under fixed weights.

Brooks et al. then connected fixed-weight contextual adaptation to reinforcement learning. In six small control tasks, their In-Context Policy Iteration algorithm appended real trajectories to an experience buffer, sampled prompt context from that buffer for a frozen model, and used model-generated rollouts for greedy action selection [9].

These results motivate a PIHF design hypothesis: a pretrained executor has task-relevant representations and knowledge that an explicit task policy can recruit to organize diagnostic behavior under fixed weights. PIHF uses this capacity as its execution substrate and makes the persistent object a versioned, expert-governed artifact  $A_t = (P_t, T_t)$ . The policy  $P_t$  supplies the textual task specification, and  $T_t$  supplies the available tools. Candidates are evaluated across the complete development panel, and each admitted artifact change guides subsequent cases. This policy-level reuse concentrates expert feedback on evaluated artifact changes and supplies PIHF’s sample-efficiency rationale. The corresponding execution and update scopes are

$$\begin{aligned} \text{execution under } A_t : \quad & \tau \sim \pi_{M,A_t}(\cdot \mid z), \quad M \text{ and } A_t \text{ fixed,} \\ \text{persistent PIHF update :} \quad & A_t \longrightarrow A_{t+1}, \quad M \text{ fixed.} \end{aligned} \tag{13}$$

The first line summarizes a rollout under a frozen artifact. Each model call conditions  $M$  on the current model-visible context assembled under  $A_t$ ; intervening tool interactions connect these calls into the complete trajectory  $\tau$ . Across panel cases,  $z$  varies; conditional on each case, trajectory sampling varies. The executor  $M$  and current artifact  $A_t$  remain fixed throughout that evaluation. The second line is persistent improvement: panel evaluation and expert admission determine whether a revised artifact becomes  $A_{t+1}$ . In-context learning names the sequence-bounded conditioning mechanism, while the versioned external artifact carries persistent state across rollouts and PIHF iterations. The development panel supplies the empirical evidence used in that admission decision.

## 5 PIHF operator and composed runs

Let

$$(P_\star, T_\star) = \text{PIHF}(M, P_{\text{init}}, T_{\text{init}}, \mathcal{D}_{\text{dev}}). \tag{14}$$

The inputs are the frozen executor  $M$ , initial policy  $P_{\text{init}}$ , initial tools  $T_{\text{init}}$ , and complete development panel  $\mathcal{D}_{\text{dev}}$ . A declared invocation also fixes its comparison protocol and stopping condition. The output  $(P_{\star}, T_{\star})$  is the final admitted artifact when that invocation ends.

The study used two composed invocations. First, public-policy development began from the initial artifact on a LIRICAL development panel:

$$(P_L, T_L) = \text{PIHF} \left( M_L, P_0, T_0, \mathcal{D}_L^{\text{dev}} \right). \quad (15)$$

Second, UDN development warm-started from the frozen LIRICAL artifact:

$$(P_U, T_U) = \text{PIHF} \left( M_U, P_L, T_L, \mathcal{D}_U^{\text{dev}} \right). \quad (16)$$

Development exclusion is policy-specific. Claims about  $(P_L, T_L)$  exclude  $\mathcal{D}_L^{\text{dev}}$  from its held-out evaluation, and claims about  $(P_U, T_U)$  exclude  $\mathcal{D}_U^{\text{dev}}$ . The warm start evaluates procedural reuse and artifact adaptation in discrete evaluations. Unchanged cross-cohort transfer is a separate estimand.

*RL parallel.* A warm start initializes a new improvement run from an admitted policy representation.

The following sections unpack the complete-panel evaluation, proposal formation, candidate comparison, admission, and stopping operations within each invocation.

## 6 Development-panel evaluation

Let the complete development panel be

$$\mathcal{D}_{\text{dev}} = \{(z_i, y_i^*)\}_{i=1}^n, \quad (17)$$

with panel composition and inference settings frozen for a declared run. During each executor rollout,  $y_i^*$  and derived outcome signals are hidden from the executor. For a frozen artifact  $A$ , define

$$\widehat{\text{Recall}}_k(M, A; \mathcal{D}_{\text{dev}}) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y_i^* \in \text{Top}_k(\widehat{y}_i(M, A))\}, \quad k \in \{1, 5\}. \quad (18)$$

Here  $y_i^*$  is the reference diagnosis for case  $z_i$ , and  $\widehat{y}_i(M, A)$  is the ranked differential produced by executor  $M$  under artifact  $A$ . The operator  $\text{Top}_k$  returns its first  $k$  diagnoses. The indicator equals one when  $y_i^*$  appears among them and zero otherwise. Thus  $\widehat{\text{Recall}}_k$  is empirical  $\text{Recall}@k$ , the fraction of the  $n$  development cases whose reference diagnosis appears within the first  $k$  positions. The hat on  $\widehat{y}_i$  marks a model-produced prediction; the hat on  $\widehat{\text{Recall}}_k$  marks the empirical value computed on the finite panel.

Because  $\text{Top}_1(\widehat{y}_i) \subseteq \text{Top}_5(\widehat{y}_i)$ , every  $\text{Recall}@1$  success is also a  $\text{Recall}@5$  success, and therefore  $\widehat{\text{Recall}}_1(M, A; \mathcal{D}_{\text{dev}}) \leq \widehat{\text{Recall}}_5(M, A; \mathcal{D}_{\text{dev}})$ .  $\text{Recall}@1$  measures correct first-rank placement, while  $\text{Recall}@5$  measures inclusion in the five-item differential.

*RL parallel.* The indicator in Equation 18 is a case-level terminal outcome signal at cutoff  $k$ . Its empirical mean is computed after a frozen candidate has executed the complete panel and serves as a post-revision validation check of diagnostic accuracy.

## 7 Critic and expert proposal formation

After executor outputs are sealed, let  $E_t$  denote the complete-panel record of trajectories, tool use, and retrospective outcomes at iteration  $t$ . A second LLM critic reviews  $E_t$  for recurrent reasoning and tool-use failures. It proposes an interpretation, localizes the implicated policy stage, and suggests evaluation measures, constraints, protected-win checks, and a candidate change to the policy, tools, or both. Denote this proposal material by  $u_t^G$ .

The expert reviews the critic’s proposal against  $E_t$  and the incumbent artifact. The expert may accept or reject the critic’s interpretation, revise it, or replace it with a new interpretation supported by the panel evidence. The expert can likewise originate or revise the hypotheses, measures, constraints, protected-win checks, and artifact changes. Write this proposal-formation step as

$$u_t = H_{\text{form}}(u_t^G, E_t, A_t), \quad (19)$$

Here  $H_{\text{form}}$  denotes expert proposal formation. Its inputs are the critic proposal  $u_t^G$ , the panel evidence  $E_t$ , and the incumbent artifact  $A_t$ . Its output  $u_t$  is the expert-authorized proposal for candidate testing;  $u_t = \emptyset$  ends that proposal path. The expert then defines the candidate protocol. Admission and rollback remain expert decisions after complete-panel evaluation.

*RL parallel.* The critic plays the generative reward model (GRM) role by turning trajectory evidence into a reasoned evaluation; PIHF also asks it to propose a correction. The expert is the human evaluator who validates or revises that feedback and controls candidate authorization, admission, and rollback.

Outcome information therefore occupies three separated roles:

1. **Executor diagnosis.** Outcomes remain hidden while the executor reasons and uses tools on each case.
2. **Outer-loop failure selection.** After outputs are sealed, outcome summaries may help the critic and expert prioritize recurrent failures and construct candidate revisions.
3. **Terminal outcome validation.** Recall@1 and Recall@5 are computed after each frozen candidate executes the complete panel. They test whether a reasoning-policy revision preserves or improves diagnostic accuracy before expert admission.

## 8 Candidate freeze and expert admission

The expert proposal  $u_t$  authorizes one edit  $\delta_t$  to the incumbent artifact  $A_t$ . Applying and freezing that edit produces the candidate

$$A'_t = \text{Freeze}(A_t \oplus \delta_t). \quad (20)$$

Here  $\oplus$  means “apply the edit.” The freeze fixes the policy and tool versions, development panel, inference settings, and output schema. The incumbent  $A_t$  and candidate  $A'_t$  are then evaluated on the complete panel under the same protocol.

The recall-preservation indicator is

$$I_t^{\text{recall}} = \mathbf{1} \left\{ \begin{array}{l} \widehat{\text{Recall}}_1(M, A'_t; \mathcal{D}_{\text{dev}}) \geq \widehat{\text{Recall}}_1(M, A_t; \mathcal{D}_{\text{dev}}) \\ \wedge \widehat{\text{Recall}}_5(M, A'_t; \mathcal{D}_{\text{dev}}) \geq \widehat{\text{Recall}}_5(M, A_t; \mathcal{D}_{\text{dev}}) \end{array} \right\}. \quad (21)$$

The indicator equals one only when both inequalities hold: the candidate preserves Recall@1 and Recall@5 relative to the incumbent. A regression in either endpoint makes the indicator zero.

The qualitative expert indicator is

$$I_t^{\text{expert}} = \mathbf{1} \left\{ \begin{array}{l} \text{clinical suggestions are sound, the revision addresses a generalizable} \\ \text{panel-supported pattern, and information-boundary rules are satisfied} \end{array} \right\}. \quad (22)$$

The expert records one binary verdict across these qualitative criteria after reviewing the candidate trajectories, complete-panel evidence, and protected prior wins. This indicator summarizes the final admission judgment; expert interpretation and proposal formation occur earlier in Equation 19.

For the current candidate comparison, the candidate becomes the next incumbent only when both indicators equal one:

$$A_{t+1} = \begin{cases} A'_t, & I_t^{\text{recall}} I_t^{\text{expert}} = 1, \\ A_t, & \text{otherwise.} \end{cases} \quad (23)$$

Equation 23 governs candidate admission. When either indicator equals zero, the incumbent remains in force while the expert rejects the proposal or revises its interpretation, measures, constraints, or edit to form a new candidate. Every revised candidate is frozen and evaluated through the same two indicators. Rollback is a separate expert action: the expert selects a previously admitted safe checkpoint, freezes it as the restored incumbent, and forms subsequent proposals from that checkpoint.

*RL parallel.* Candidate formation is the policy-improvement step. Complete-panel execution supplies post-revision outcome evidence, and expert admission determines whether the proposed policy representation persists.

## 9 Process-guided policy improvement and outcome validation

Research on language-model reasoning distinguishes process-based feedback, which evaluates the reasoning process, from outcome-based feedback, which evaluates the final result. Uesato et al. instantiate this distinction within expert-iteration RL, treating each generated reasoning step as an action and comparing policy-improvement procedures driven by final-answer correctness, outcome-supervised reward models, and process-supervised reward models [10]. Lightman et al. compare process- and outcome-supervised reward models for best-of- $N$  selection with a fixed generator. Their process labels identify the first incorrect step, providing more precise feedback and easing the reward model’s credit-assignment problem [11].

PIHF applies this functional separation at policy-stage and complete-trajectory granularity. The critic and expert inspect sealed reasoning and tool-use trajectories, localize a recurrent failure to the implicated stage or tool behavior, interpret its cause, and form a targeted revision of  $A_t$ . The expert may replace the critic’s interpretation and originate new proposal elements from the panel evidence. This stage-localized review assigns credit to the external policy component that should change and drives the policy-improvement step in Equation 19.

Recall@1 and Recall@5 enter after the frozen candidate executes the complete panel. They are terminal outcome validation metrics: Recall@1 measures correct first-rank placement, and Recall@5 measures inclusion within the five-item differential. The recall-preservation indicator in Equation 21 tests whether the process-guided revision preserves diagnostic accuracy. Improvement in either metric provides downstream evidence of better diagnostic performance. PIHF therefore improves the reasoning and tool-use policy through process feedback, then validates the clinical outcome of that revision with Recall@1 and Recall@5.

Each PIHF invocation declares its stopping condition in the run protocol. In the reported development study, iteration ended after diagnostic performance had plateaued for 10 or more completed iterations under finite compute [12]. This is the recorded stopping observation for that study invocation. A new invocation specifies its stopping condition before candidate evaluation begins. The last admitted artifact is then frozen for development-excluded evaluation.

*RL parallel.* Process feedback supplies stage-localized credit assignment for policy improvement. Recall@1 and Recall@5 supply terminal outcome validation. Expert admission commits the resulting external policy as the next incumbent.

## 10 Portability, invariance, and attribution

Let  $M_m$  index executor backbones, let  $A_\star$  be one frozen composite artifact, and let  $A_\emptyset$  denote the matched no-artifact condition. For endpoint  $k \in \{1, 5\}$ , define the within-backbone benefit

$$\Delta_{m,k} = \widehat{\text{Recall}}_k(M_m, A_\star; \mathcal{D}_{\text{transfer}}) - \widehat{\text{Recall}}_k(M_m, A_\emptyset; \mathcal{D}_{\text{transfer}}). \quad (24)$$

Positive  $\Delta_{m,k}$  across the declared backbones supports portability of the frozen artifact. A separate invariance estimand is the dispersion

$$\text{Disp}_k = \max_m \Delta_{m,k} - \min_m \Delta_{m,k}. \quad (25)$$

Low dispersion supports similarity of benefit across backbones. Both estimands permit backbone-specific trajectories and absolute performance. Portability and invariance therefore require separate claims and separate uncertainty analyses.

*RL parallel.* Holding  $A_\star$  fixed while changing  $M_m$  evaluates how the same policy representation induces behavior under different executors.

The identified object is the composite policy-and-tool artifact because policy content, tool access, and their coordination differ between  $A_\star$  and  $A_\emptyset$ . Current transfer evidence supports the composite artifact. Stronger attribution to the optimized reasoning process requires a content-only versus with-critique ablation that separates expert-written clinical content from the iteratively revised process structure.

## 11 Summary

PIHF persistently improves a versioned in-context policy-and-tool artifact  $A_t = (P_t, T_t)$  executed by a frozen pretrained model. A language-model critic reviews complete-panel reasoning and tool-use trajectories to identify recurrent failures and propose interpretations and changes. The expert can accept, revise, replace, or originate these interpretations and changes and retains authority over candidate formation, admission, and rollback. Stage-localized process feedback directs revision, while Recall@1 and Recall@5 validate diagnostic outcomes after each frozen candidate executes.

In the liteOdyssey study, a PIHF-derived policy developed from 50 cases and executed with its tool set increased Recall@1 from 26.5% to 59.3% across 1,243 public rare-disease benchmark cases in the frontier proprietary executor and transferred across proprietary and open-weight executors [12]. For rare-disease diagnosis, PIHF converted scarce expert reasoning into an inspectable, revisable execution policy that could be reused across model backbones while model weights remained fixed.



## References

- [1] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, November 2018. ISBN 9780262039246.
- [2] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-Tuning Language Models from Human Preferences. 2019. doi: 10.48550/ARXIV.1909.08593. URL <https://arxiv.org/abs/1909.08593>.
- [3] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022. doi: 10.52202/068431-2011. URL <https://arxiv.org/abs/2203.02155>.
- [4] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.48550/ARXIV.2305.18290. URL <https://arxiv.org/abs/2305.18290>.
- [5] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/bf00992696. URL <http://dx.doi.org/10.1007/BF00992696>.
- [6] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving Language Understanding by Generative Pre-Training. Technical report, OpenAI, June 2018. URL [https://cdn.openai.com/research-covers/language-unsupervised/language\\_understanding\\_paper.pdf](https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf).
- [7] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. Technical Report GPT-2 Technical Report, OpenAI, February 2019. URL [https://cdn.openai.com/better-language-models/language\\_models\\_are\\_unsupervised\\_multitask\\_learners.pdf](https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf).
- [8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33*, 2020. URL <https://arxiv.org/abs/2005.14165>.
- [9] Ethan Brooks, Logan Walls, Richard L. Lewis, and Satinder Singh. Large Language Models can Implement Policy Iteration. 2022. doi: 10.48550/ARXIV.2210.03821. URL <https://arxiv.org/abs/2210.03821>.
- [10] Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. 2022. doi: 10.48550/ARXIV.2211.14275. URL <https://arxiv.org/abs/2211.14275>.
- [11] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s Verify Step by Step. 2023. doi: 10.48550/ARXIV.2305.20050. URL <https://arxiv.org/abs/2305.20050>.
- [12] Minh-Ha Nguyen, Erica Gray, Bryce A. Schuler, Kevin W. Byram, Chih-Ting Yang, Fan Ma, Hua Xu, Wu-Chen Su, Chao Yan, Wei-Qi Wei, Adam Wright, Lisa Bastarache, Josh Peterson, Lingyao Li,

Siyuan Ma, Undiagnosed Diseases Network, Rizwan Hamid, Thomas A. Cassini, and Cathy Shyr. Teaching agentic AI to learn expert reasoning for rare disease diagnosis. 2026. doi: 10.48550/ARXIV.2606.16149. URL <https://arxiv.org/abs/2606.16149>.